

Изменение значений:

- `array_map` не может изменять значения внутри входных массивов, а `array_walk` - может; в частности, `array_map` никогда не меняет своих аргументов.

Доступ к ключам доступа:

- `array_map` не может работать с ключами массива, `array_walk` - может.

Возвращаемое значение:

- `array_map` возвращает новый массив, `array_walk` возвращает `true/false`. Следовательно, если вы не хотите создавать массив в результате прохождения одного массива, вы должны использовать его `array_walk`.

Итерация нескольких массивов:

- `array_map` также может принимать произвольное количество массивов, и он может перебирать их параллельно, а `array_walk` работает только на одном.

Передача произвольных данных в обратный вызов:

- `array_walk` может получить дополнительный произвольный параметр для передачи обратного вызова.

Длина возвращаемого массива:

- Результирующий массив `array_map` имеет ту же длину, что и наибольший входной массив.
- `array_walk` не возвращает массив, но в то же время он не может изменить количество элементов исходного массива;

Пример:

```
<pre>
<?php

$origarray1 = array(2.4, 2.6, 3.5);
$origarray2 = array(2.4, 2.6, 3.5);

print_r(array_map('floor', $origarray1)); // $origarray1 stays the same

// changes $origarray2
array_walk($origarray2, function (&$v, $k) { $v = floor($v); });
print_r($origarray2);

// this is a more proper use of array_walk
array_walk($origarray1, function ($v, $k) { echo "$k => $v", "\n"; });

// array_map accepts several arrays
print_r(
    array_map(function ($a, $b) { return $a * $b; }, $origarray1, $origarray2)
);

// select only elements that are > 2.5
print_r(
    array_filter($origarray1, function ($a) { return $a > 2.5; })
);

?>
</pre>
```

Результат:

Array

```
(  
  [0] => 2  
  [1] => 2  
  [2] => 3  
)
```

Array

```
(  
  [0] => 2  
  [1] => 2  
  [2] => 3  
)
```

0 => 2.4

1 => 2.6

2 => 3.5

Array

```
(  
  [0] => 4.8  
  [1] => 5.2  
  [2] => 10.5  
)
```

Array

```
(  
  [1] => 2.6  
  [2] => 3.5  
)
```